

Kubernetes Incident Lab — Run Report

PV failure + DNS recovery on minikube — as actually run

minikube · v1.34.0

This report documents a real end-to-end incident-response exercise on a local Kubernetes cluster: injecting a storage fault (a PersistentVolumeClaim that cannot bind), observing it through Prometheus telemetry, and recovering it — including a botched first recovery that had to be diagnosed and corrected. It reflects what the environment actually did, on Windows/PowerShell with a kube-prometheus-stack install, rather than an idealized script.

Environment

minikube v1.34.0 (Docker driver) on Windows · kubectl via PowerShell
kube-prometheus-stack (Helm release "kps") in the monitoring namespace
Victim app in the app namespace: frontend (2 replicas) + backend + PVC

1 Environment and baseline

The cluster came up with one node in the control-plane role. Namespaces were created to separate the workload under test from the telemetry watching it: `app` for the victim application, `monitoring` for Prometheus and Grafana. Keeping them apart means the injected faults hit `app` while the monitoring stack stays healthy and keeps observing.

```
kubectl get nodes
# NAME          STATUS    ROLES          AGE   VERSION
# minikube     Ready     control-plane  5m    v1.34.0
kubectl create namespace app
kubectl create namespace monitoring
```

2 Telemetry discovery — reading the real targets

The written plan assumed a Prometheus scrape job named `kube-dns`. On this install that job does not exist, so `up{job="kube-dns"}` returned no data. The fix was to query bare `up` and read the actual target list — a small but real lesson: verify label names against the running system, don't trust them from a doc.

```
up{job="kube-dns"}
# Empty query result -> label name is wrong on this install

up
# lists every scrape target with its real job= label
```

Control-plane targets reading 0 are normal on minikube

up == 0 for kube-proxy, kube-scheduler, kube-etcd, kube-controller-manager. Those components bind metrics to localhost inside the node, so Prometheus cannot scrape them from outside. The components are healthy; only their metrics are unreachable. Unrelated to the faults in this lab.

3 An unexpected finding — DNS already down

While checking CoreDNS, the replica-availability metric returned 0. Cross-checking with `kubectl` showed CoreDNS at `0/0` with an age of 41h — meaning DNS had been scaled to zero long before the lab began, not by a lab step. Rather than treat this as the injected DNS fault, the cluster was restored to a clean baseline first so the storage fault could be studied in isolation.

```
kubectl -n kube-system get deploy coredns
# READY 0/0   AVAILABLE 0   AGE 41h   <- pre-existing, not a lab step
kubectl -n kube-system scale deploy/coredns --replicas=2
# READY 2/2   AVAILABLE 2   <- clean baseline restored
```

Telemetry lesson

A metric reading 0 (or a query returning empty) is only meaningful once you confirm what it measures still exists and is in the expected shape. The number was accurate; the assumption that the baseline was healthy was not.

4 Injecting the storage fault

The fault: point a fresh PVC at a StorageClass that does not exist on minikube (`gp3-fast` ; the only real class is `standard`). The claim can never bind, so the backend Pod that mounts it can never schedule. Because this ran under PowerShell, the bash here-doc from the original plan was replaced with a PowerShell here-string (`@' ... '@`).

```
kubectl -n app scale deploy/backend --replicas=0
kubectl -n app delete pvc data-pvc

@'
apiVersion: v1
kind: PersistentVolumeClaim
metadata: { name: data-pvc, namespace: app }
spec:
  accessModes: [ReadWriteOnce]
  storageClassName: gp3-fast          # does not exist -> cannot bind
  resources: { requests: { storage: 512Mi } }
'@ | Out-File -FilePath broken-pvc.yaml -Encoding utf8

kubectl apply -f broken-pvc.yaml
kubectl -n app scale deploy/backend --replicas=1
```

5 Observing the fault — two views agree

kubectl showed the exact signature of a storage/scheduling fault: the PVC `Pending` on class `gp3-fast`, the backend Pod `Pending` with `PodScheduled: False`, and the textbook scheduler event.

```
kubectl -n app get pods,pvc
# pod/backend-... 0/1 Pending
# pod/frontend-... 1/1 Running (x2) <- unaffected
# pvc/data-pvc Pending STORAGECLASS gp3-fast

kubectl -n app describe pod -l app=backend # Events:
# FailedScheduling: pod has unbound immediate PersistentVolumeClaims
```

The same fault, seen through Prometheus — each query returned one series with value 1, naming the exact stuck objects:

```
kube_persistentvolumeclaim_status_phase{namespace="app",phase="Pending"} == 1
# persistentvolumeclaim="data-pvc" -> 1
kube_pod_status_phase{namespace="app",phase="Pending"} == 1
# pod="backend-95d478496-..." -> 1
```

The triage tell

Backend is Pending (never scheduled), not CrashLoopBackOff -> storage/scheduling, not an app bug. Both frontends stay Running -> blast radius is the one workload that claims the volume, not the namespace. Rule: classify by Pod phase first, and never patch the frontend when the fault is beneath it.

6 Recovery — and catching a botched one

The fix is to recreate the PVC against the real `standard` class. On the first attempt the delete and scale-down ran, but the recreate did not take — leaving the PVC and backend Pod deleted entirely. The Prometheus queries went empty, which looked like success but was empty for the wrong reason: nothing existed to be Pending. Cross-checking `kubectl` exposed it.

```
kubectl -n app get pvc data-pvc
# Error from server (NotFound): ... "data-pvc" not found
kubectl -n app get pods -l app=backend
# No resources found in app namespace.

kubectl -n app get pods,pvc,deploy
# deployment.apps/backend 0/0 <- Deployment survived, just scaled to 0
```

Because the backend Deployment still existed at `0/0`, the real recovery was to recreate the PVC (correct class) and scale the backend back up:

```
@'
apiVersion: v1
kind: PersistentVolumeClaim
metadata: { name: data-pvc, namespace: app }
spec:
  accessModes: [ReadWriteOnce]
  storageClassName: standard      # the real, existing class
  resources: { requests: { storage: 512Mi } }
'@ | Out-File -FilePath fixed-pvc.yaml -Encoding utf8

kubectl apply -f fixed-pvc.yaml
kubectl -n app scale deploy/backend --replicas=1
```

Verification — now empty for the right reason, with the objects present and healthy:

```
kubectl -n app get pvc data-pvc
# data-pvc Bound pvc-b225... 512Mi RW0 standard
kubectl -n app get pods -l app=backend
# backend-... 1/1 Running

# Prometheus: both Pending queries -> Empty query result (0 series)
```

7 What this exercise demonstrates

Competency	Evidence from this run
Systematic triage	Classified by Pod phase; isolated fault to one workload
Observability fluency	PromQL for PVC/Pod phase; discovered real scrape labels from bare up
Environment adaptation	Translated bash here-docs to PowerShell here-strings
Healthy skepticism	Caught empty-for-wrong-reason; verified metrics against <code>kubectl</code>

Recovery discipline	Diagnosed a failed recovery, restored from actual cluster state
---------------------	---

One-line takeaway

An empty or zero telemetry signal means "healthy" only after you confirm the measured object still exists. Detect with metrics; confirm root cause with kubectl; match the fix to the fault — and verify the fix the same two ways you found it.

© 2026 David Grunwald. All rights reserved.